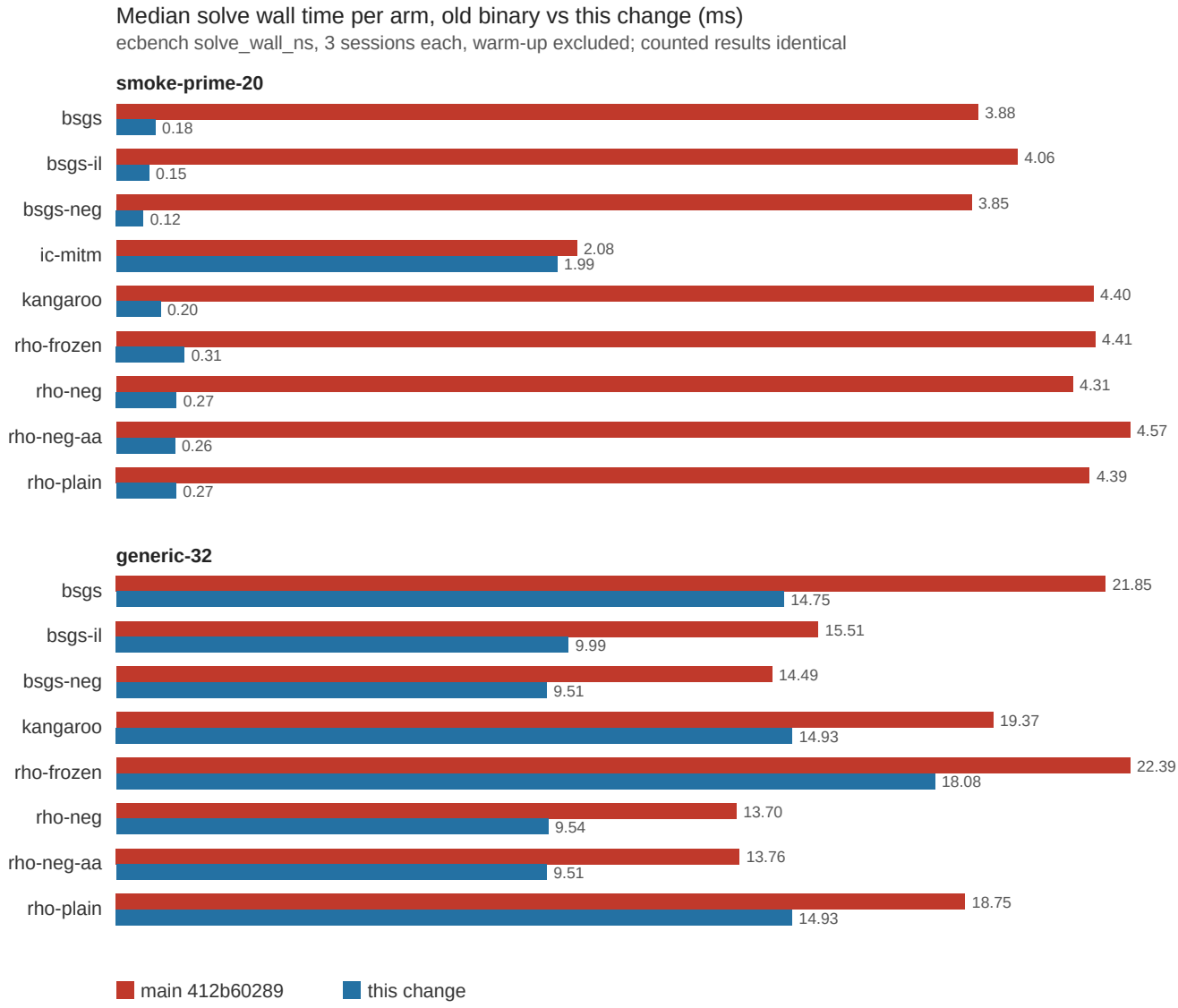
ecbench harness overhead: parse the registry once, allocate counter keys once

Two harness changes. `docs/curves/registry.json` (4.7 MB) is parsed once per process instead of on every registry lookup (`Workload::on` in every measured child and every planned target). `GenericOutcome::count`, called several times per BSGS and kangaroo step, allocates a counter's key only on first use. No solver, cost unit or counted result changes: 2376 records are identical to the old binary's outside timing, every session replays exactly under the other binary (`ecbench verify --replay-all`), and the §5a band is identical.



Old binary vs this change

Median over 3 sessions per binary, alternated old/new, warm-up excluded. *solve* is the method's timed window (`solve_wall_ns`); *process* is the whole measured child.

spec	arm	n	solve old ms	solve new ms	new/old	process old ms	process new ms	new/old
smoke-prime-20	bsgs	18	3.884	0.176	0.045	34.2	26.8	0.784
smoke-prime-20	bsgs-il	18	4.062	0.148	0.037	34.5	26.9	0.779
smoke-prime-20	bsgs-neg	18	3.854	0.122	0.032	34.6	26.6	0.768
smoke-prime-20	ic-mitm	18	2.076	1.987	0.957	41.5	33.5	0.806
smoke-prime-20	kangaroo	18	4.404	0.200	0.045	34.6	26.6	0.768

smoke-prime-20	rho-frozen	18	4.412	0.307	0.070	34.5	27.1	0.785
smoke-prime-20	rho-neg	18	4.311	0.272	0.063	34.4	26.7	0.776
smoke-prime-20	rho-neg-aa	18	4.569	0.265	0.058	35.1	26.4	0.753
smoke-prime-20	rho-plain	18	4.387	0.270	0.062	34.5	26.5	0.769
generic-32	bsgs	45	21.855	14.747	0.675	55.9	46.1	0.825
generic-32	bsgs-il	45	15.507	9.991	0.644	49.5	38.9	0.785
generic-32	bsgs-neg	45	14.486	9.509	0.656	47.5	37.7	0.794
generic-32	kangaroo	45	19.367	14.934	0.771	51.5	43.0	0.834
generic-32	rho-frozen	45	22.391	18.084	0.808	57.0	47.4	0.831
generic-32	rho-neg	45	13.696	9.540	0.697	47.1	38.1	0.808
generic-32	rho-neg-aa	45	13.763	9.509	0.691	46.9	37.4	0.799
generic-32	rho-plain	45	18.752	14.928	0.796	51.7	43.8	0.847

Which change does what

Median solve ms, one binary per change, 2 sessions each. The registry change removes 3.5–4 ms from every timed solve window and about 7 ms from every measured child; the counter change removes 1.8–2.8 ms from each BSGS arm at 32 bits (no visible effect at 20 bits); a word Euclid inverse in `prime_fast`, 2.3–2.9× faster per inversion in isolation, moved nothing and is not part of the change.

spec	arm	old	registry once	counter keys	Euclid inverse	all three
smoke-prime-20	bsgs	3.54	0.21	3.66	3.54	0.18
smoke-prime-20	bsgs-il	3.62	0.18	3.76	3.74	0.15
smoke-prime-20	bsgs-neg	3.54	0.13	3.70	3.54	0.12
smoke-prime-20	ic-mitm	1.98	1.90	2.01	1.91	2.03
smoke-prime-20	kangaroo	4.10	0.21	4.35	3.97	0.20
smoke-prime-20	rho-frozen	4.13	0.30	4.29	4.09	0.32
smoke-prime-20	rho-neg	4.19	0.26	4.26	4.03	0.28
smoke-prime-20	rho-neg-aa	4.07	0.25	4.21	4.04	0.26
smoke-prime-20	rho-plain	4.09	0.30	4.19	4.17	0.29
generic-32	bsgs	20.20	19.04	17.39	21.02	14.51
generic-32	bsgs-il	14.51	14.39	12.69	15.39	9.74
generic-32	bsgs-neg	13.97	10.97	11.94	13.38	9.11
generic-32	kangaroo	18.78	15.12	18.59	19.07	15.30
generic-32	rho-frozen	22.11	17.96	21.57	21.79	17.14
generic-32	rho-neg	12.87	9.40	13.30	12.62	9.17
generic-32	rho-neg-aa	13.09	9.38	12.96	12.96	9.09
generic-32	rho-plain	18.45	14.03	18.40	17.89	14.71

Why the solve window was affected

Nothing in the solve path reads the registry. Callgrind counts the same instructions inside the window for both binaries (1.110 M vs 1.112 M on a 20-bit `rho.negation` solve), and the old child executes 44 M more instructions before it (193 M vs 149 M: the second parse). The old binary drops its freshly parsed copy of the registry immediately before the solve; the 3.5–4 ms it then loses inside the window is wall time with no extra instructions and no syscalls. Disabling glibc fast bins did not remove it; the exact memory-system cause was not isolated. Wall times recorded by the old binary on small curves therefore carry this offset; counted results do not.

Host: Intel Xeon @ 2.10 GHz, 4 vCPU cloud container; sessions pinned by ecbench (`--cpus auto`) with `--allow-busy`, so below L2. Engineering change to harness overhead, not an index-calculus result.